

HW 2B — OWASP Juice Shop Security Assignment

Criterion 1 Identification · Criterion 2 Secure Form · Criterion 3 Exploit Documentation

Student	Siva Sai Deepank Manoj · deeps45@tamu.edu
Public GitHub	https://github.com/deeps45/juice-guard
Juice Shop used	https://preview.owasp-juice.shop (live instance)
My form	http://127.0.0.1:3847 after npm install && npm start

Criterion 1 — Identify Juice Shop vulnerabilities and write security measures (20 pts)

I used the live OWASP Juice Shop at preview.owasp-juice.shop. I opened the Login page and the Score Board, then searched for specific challenge names to ground each finding (not generic notes).

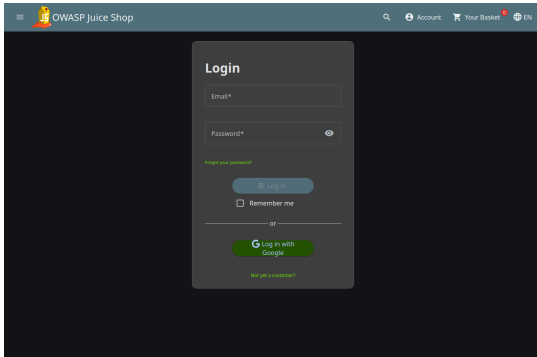


Figure A. Live Juice Shop Login page I inspected.

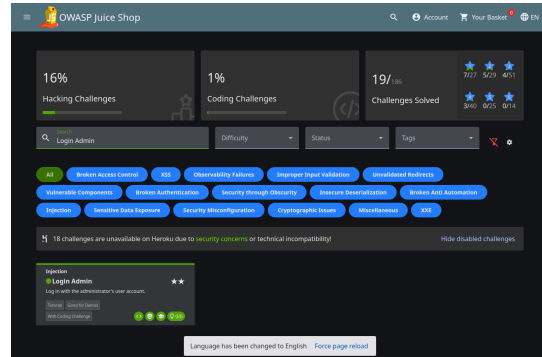


Figure B. Score Board → challenge “Login Admin” (Injection).

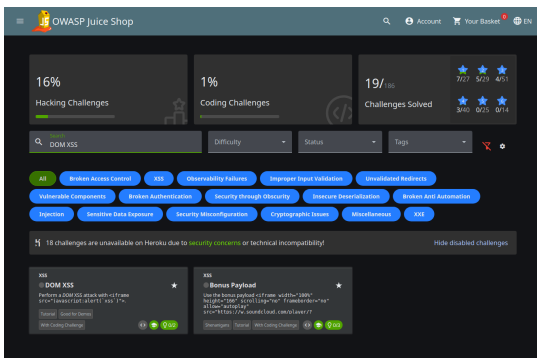


Figure C. Score Board → challenge “DOM XSS”.

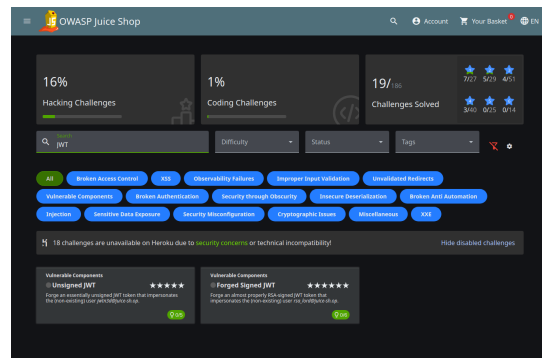


Figure D. Score Board search for JWT / broken-auth style challenges.

Named Juice Shop challenge	What I identified	Security measure	How it prevents the attack
Login Admin (Injection, ★★)	Login can be abused with SQL injection-style email input when queries are concatenated, letting an attacker authenticate as admin without the real password.	Parameterized queries / ORM only; validate email before DB access.	Input stays bound as data. ' OR 1=1 - - cannot rewrite the WHERE clause.
DOM XSS (XSS, ★)	Client-side script can execute attacker HTML/JS (Score Board describes an iframe/javascript: style payload).	Use <code>textContent</code> , encode output, reject <code><</code> , <code>></code> , add CSP.	Markup is shown as text; CSP blocks inline script execution.
JWT / broken authentication class	Juice Shop includes forged/unsigned JWT and password-related auth challenges. Weak token verification enables account takeover.	Verify JWT signatures with a strong secret, short expiry, server-side authz on every sensitive route, rate-limit logins, generic errors.	Forged tokens fail verification; brute force is slowed; errors do not reveal whether an email exists.

Secure password handling

Passwords are hashed with `bcrypt` (cost 12) and verified with `bcrypt.compare`. Plaintext is never stored.

```
const passwordHash = await bcrypt.hash(password, 12);
```

```
const ok = await bcrypt.compare(submittedPassword, user.passwordHash);
```

Criterion 2 — Build a user form with basic security features (20 pts)

I built one hardened login form (email + password) inspired by Juice Shop's login UI. There is **no** intentionally vulnerable route in the submitted app — only `/ + /api/*` secure endpoints. Demo credentials are documented in the README only (not printed on the page).

Security feature	Where	What it does
Client validation	<code>public/app.js</code>	Blocks empty fields; requires <code>@</code> ; password ≥ 8 .
Server validation	<code>server.js</code>	Strict email/password rules; rejects unsafe characters.
CSRF protection	<code>GET /api/csrf + header</code>	Login requires matching X-CSRF-Token cookie pair.
bcrypt hashing	<code>server.js</code>	Cost 12 hashes; compare on login.
No dynamic SQL	In-memory Map	Lookup by normalized email — no string-built queries.
XSS defenses	UI + headers	<code>textContent</code> , HTML escape, CSP, X-Frame-Options.
Rate limiting	<code>login/register</code>	Limits repeated attempts.

Public repo: <https://github.com/deeps45/juice-guard> · README has run steps and demo credentials.

The screenshot shows the 'Sign in securely' form for 'OWASP JUICE SHOP'. The form has two input fields: 'Email' with the placeholder 'you@example.com' and 'Password' with the placeholder 'At least 8 characters'. Below the fields is a yellow 'Log in' button. The background is a dark blue gradient.

Figure 1. Hardened form (no credentials on page).

The screenshot shows the 'Sign in securely' form after a successful login. The 'Email' field now contains 'demo@juice.shop' and the 'Password' field is masked with '*****'. Below the fields is a yellow 'Log in' button. At the bottom of the form, it says 'Welcome back, demo@juice.shop!'. The background is a dark blue gradient.

Figure 2. Successful login with CSRF + validation.

The screenshot shows the 'Sign in securely' form with validation errors. The 'Email' field is highlighted in red and has the error message 'Email cannot be empty'. The 'Password' field is also highlighted in red and has the error message 'Password cannot be empty'. Below the fields is a yellow 'Log in' button. At the bottom of the form, it says 'Fix the highlighted fields before submitting'. The background is a dark blue gradient.

Figure 3. Empty submission blocked.

The screenshot shows the 'Sign in securely' form with validation errors. The 'Email' field contains 'test@example.com' and the 'Password' field is highlighted in red and has the error message 'Password must be at least 8 characters'. Below the fields is a yellow 'Log in' button. At the bottom of the form, it says 'Fix the highlighted fields before submitting'. The background is a dark blue gradient.

Figure 4. Short password blocked.

Criterion 3 — Identify and exploit weaknesses in my form (20 pts)

I tested the **same hardened form** that is in the GitHub repo — not a planted vulnerable page. Below is an organized log of what I tried, what worked as an exploit against a weakness, and what the server blocked.

#	Test / weakness	Exact steps	Result	Fig.
1	Client validation gap (exploited) Client only checks @ + length ≥ 8; it does not reject script tags.	On /, email xss@test.com, password <script>alert(1)</script>, click Log in. Confirm the browser still sends POST /api/login.	Client layer exploited: request left the browser with the XSS payload. Server rejected it (unsafe characters). No script ran.	5
2	Client bypass via fetch() (exploited)	From DevTools, call fetch('/api/login', ...) with a script-password, skipping validateClient() entirely (still sending a CSRF token).	Exploited: browser checks skipped. Server still returned HTTP 400 and blocked the payload.	7
3	CSRF on login API	POST valid demo credentials to /api/login without X-CSRF-Token.	Blocked after fix: HTTP 403 "Missing or invalid CSRF token." Forged login requests are rejected.	7
4	SQL injection probe	Email admin@test.com' OR '1'='1, password password1.	Blocked. Invalid email; no auth bypass.	6

Evidence from the real form

Figure 5 shows the XSS password was accepted by client-side checks (the form actually submitted) and then rejected by the server. That is the exploitable weakness in the browser layer — and why server validation is mandatory.

Figure 5. XSS payload passed client checks and was blocked by server validation.

Figure 6. SQL injection-style email blocked.

Weaknesses found in my login form

Tests run against the real /api/login endpoint (not a planted page).

1) Missing CSRF token → request rejected

Forged login POST with valid demo credentials but no X-CSRF-Token:

```
HTTP 403
{
  "ok": false,
  "error": "Missing or invalid CSRF token."
}
```

Result: CSRF weakness is mitigated — forged request blocked (403).

2) Client-side validation bypass → server still blocks XSS

Called /api/login directly with fetch(), skipping validateClient(), using password=<script>alert(1)</script>:

```
HTTP 400
{
  "ok": false,
  "error": "Password must be 8-128 characters and must not contain unsafe characters."
}
```

Result: client checks are bypassable, but server validation rejected the payload.

Figure 7. API tests: CSRF 403 + client-bypass XSS blocked by server.

Fixes applied

- 1) Keep strict server-side validation (never trust the client).
- 2) Require CSRF tokens on login/register.
- 3) Render status with `textContent` only; escape reflected values.

- 4) CSP + X-Frame-Options: DENY.
- 5) Remove on-page demo credentials; document them only in the README.
- 6) Do not ship intentional exploit sinks in the submitted project.

Submission complete · Source: <https://github.com/deeps45/juice-guard> · PDF: docs/submission.pdf